

Query Type	SQL Query	ORM Equivalent (C# Entity Framework)
Insert	INSERT INTO Users (Username, Email, Role) VALUES ('Ali', 'ali@gmail.com', 'Admin')	db.Users.Add(new User { Username = "Ali", Email = "ali@gmail.com", Role = "Admin" }); db.SaveChanges();
Insert	INSERT INTO Orders (UserId, OrderDate, Status) VALUES (1, GETDATE(), 'Pending')	db.Orders.Add(new Order { UserId = 1, OrderDate = DateTime.Now, Status = "Pending" }); db.SaveChanges();
Insert	INSERT INTO OrderItems (OrderId, ProductName, Quantity, Price) VALUES (1, 'Laptop', 1, 50000)	db.OrderItems.Add(new OrderItem { OrderId = 1, ProductName = "Laptop", Quantity = 1, Price = 50000 }); db.SaveChanges();
Update	UPDATE Users SET Role = 'User' WHERE UserId = 1	var user = db.Users.Find(1); user.Role = "User"; db.SaveChanges();
Update	UPDATE Orders SET Status = 'Completed' WHERE OrderId = 2	var order = db.Orders.Find(2); order.Status = "Completed"; db.SaveChanges();
Update	UPDATE OrderItems SET Price = 45000 WHERE OrderItemId = 3	var item = db.OrderItems.Find(3); item.Price = 45000; db.SaveChanges();
Delete	DELETE FROM Users WHERE UserId = 1	var user = db.Users.Find(1); db.Users.Remove(user); db.SaveChanges();
Delete	DELETE FROM Orders WHERE OrderId = 2	var order = db.Orders.Find(2); db.Orders.Remove(order); db.SaveChanges();
Delete	DELETE FROM OrderItems WHERE OrderItemId = 3	var item = db.OrderItems.Find(3); db.OrderItems.Remove(item); db.SaveChanges();
Select	SELECT * FROM Users WHERE Username = 'Ali'	var user = db.Users.Where(u => u.Username == "Ali").ToList();

#	SQL Query	Entity Framework (MVC.NET)
1	INSERT INTO Users (Username, Email, Role) VALUES ('john', 'john@example.com', 'Admin')	db.Users.Add(new User { Username = "john", Email = "john@example.com", Role = "Admin" }); db.SaveChanges();
2	INSERT INTO Orders (UserId, OrderDate, Status) VALUES (1, GETDATE(), 'Pending')	db.Orders.Add(new Order { UserId = 1, OrderDate = DateTime.Now, Status = "Pending" }); db.SaveChanges();
3	INSERT INTO OrderItems (OrderId, ProductName, Quantity, Price) VALUES (1, 'Keyboard', 2, 499.99)	db.OrderItems.Add(new OrderItem { OrderId = 1, ProductName = "Keyboard", Quantity = 2, Price = 499.99M }); db.SaveChanges();
4	INSERT INTO Users (Username, Email, Role) VALUES ('alice', 'alice@gmail.com', 'User')	db.Users.Add(new User { Username = "alice", Email = "alice@gmail.com", Role = "User" }); db.SaveChanges();
5	INSERT INTO Orders (UserId, OrderDate, Status) VALUES (2, '2024-01-01', 'Shipped')	db.Orders.Add(new Order { UserId = 2, OrderDate = new DateTime(2024, 1, 1), Status = "Shipped" }); db.SaveChanges();

#	SQL Query	Entity Framework (MVC.NET)
6	UPDATE Users SET Email = 'newjohn@example.com' WHERE UserId = 1	var user = db.Users.Find(1); user.Email = "newjohn@example.com"; db.SaveChanges();
7	UPDATE Orders SET Status = 'Completed' WHERE OrderId = 1	var order = db.Orders.Find(1); order.Status = "Completed"; db.SaveChanges();
8	UPDATE OrderItems SET Price = 549.99 WHERE ProductName = 'Keyboard'	var item = db.OrderItems.FirstOrDefault(o => o.ProductName == "Keyboard"); item.Price = 549.99M; db.SaveChanges();
9	UPDATE Users SET Role = 'Moderator' WHERE Username = 'alice'	var user = db.Users.FirstOrDefault(u => u.Username == "alice"); user.Role = "Moderator"; db.SaveChanges();
10	UPDATE OrderItems SET Quantity = 5 WHERE OrderItemId = 3	var item = db.OrderItems.Find(3); item.Quantity = 5; db.SaveChanges();

#	SQL Query	Entity Framework (MVC.NET)
11	DELETE FROM Users WHERE UserId = 1	var user = db.Users.Find(1); db.Users.Remove(user); db.SaveChanges();
12	DELETE FROM Orders WHERE OrderId = 2	var order = db.Orders.Find(2); db.Orders.Remove(order); db.SaveChanges();
13	DELETE FROM OrderItems WHERE OrderItemId = 4	var item = db.OrderItems.Find(4); db.OrderItems.Remove(item); db.SaveChanges();
14	DELETE FROM Users WHERE Email = 'test@example.com'	var user = db.Users.FirstOrDefault(u => u.Email == "test@example.com"); db.Users.Remove(user); db.SaveChanges();
15	DELETE FROM Orders WHERE UserId = 1 AND Status = 'Cancelled'	var orders = db.Orders.Where(o => o.UserId == 1 && o.Status == "Cancelled").ToList(); db.Orders.RemoveRange(orders); db.SaveChanges();

#	SQL Query	Entity Framework (MVC.NET)
16	SELECT * FROM Users WHERE UserId = 1	var user = db.Users.FirstOrDefault(u => u.UserId == 1);
17	SELECT * FROM Orders WHERE Status = 'Completed' AND OrderDate > '2025-01-01'	var orders = db.Orders.Where(o => o.Status == "Completed" && o.OrderDate > new DateTime(2025, 1, 1)).ToList();
18	SELECT * FROM OrderItems WHERE Price < 100	var items = db.OrderItems.Where(o => o.Price < 100).ToList();
19	SELECT * FROM Users WHERE Username LIKE 'John%'	var users = db.Users.Where(u => u.Username.StartsWith("John")).ToList();
20	SELECT * FROM Orders WHERE OrderDate BETWEEN '2025-01-01' AND '2025-12-31'	var orders = db.Orders.Where(o => o.OrderDate >= new DateTime(2025, 1, 1) && o.OrderDate <= new DateTime(2025, 12, 31)).ToList();

#	SQL Query	Entity Framework (MVC.NET)
21	SELECT Users.Username, Orders.OrderDate FROM Users JOIN Orders ON Users.UserId = Orders.UserId WHERE Orders.Status = 'Pending'	var data = db.Users.Join(db.Orders, u => u.UserId, o => o.UserId, (u, o) => new { u.Username, o.OrderDate }).Where(x => x.Status == "Pending").ToList();
22	SELECT OrderItems.ProductName, Orders.OrderDate FROM OrderItems JOIN Orders ON OrderItems.OrderId = Orders.OrderId WHERE OrderItems.Price > 50	var data = db.OrderItems.Join(db.Orders, oi => oi.OrderId, o => o.OrderId, (oi, o) => new { oi.ProductName, o.OrderDate }).Where(x => x.Price > 50).ToList();
23	SELECT Users.Username, OrderItems.ProductName FROM Users JOIN Orders ON Users.UserId = Orders.UserId JOIN OrderItems ON Orders.OrderId = OrderItems.OrderId WHERE OrderItems.Price < 100	var data = db.Users.Join(db.Orders, u => u.UserId, o => o.UserId, (u, o) => new { u.Username, o.OrderId }).Join(db.OrderItems, o => o.OrderId, oi => oi.OrderId, (o, oi) => new { o.Username, oi.ProductName }).Where(x => x.Price < 100).ToList();
24	SELECT Orders.OrderId, Users.Username FROM Orders JOIN Users ON Orders.UserId = Users.UserId WHERE Orders.OrderDate > '2025-01-01'	var data = db.Orders.Join(db.Users, o => o.UserId, u => u.UserId, (o, u) => new { o.OrderId, u.Username }).Where(x => x.OrderDate > new DateTime(2025, 1, 1)).ToList();
25	SELECT OrderItems.ProductName, Users.Username FROM OrderItems JOIN Orders ON OrderItems.OrderId = Orders.OrderId JOIN Users ON Orders.UserId = Users.UserId WHERE OrderItems.Quantity > 2	var data = db.OrderItems.Join(db.Orders, oi => oi.OrderId, o => o.OrderId, (oi, o) => new { oi.ProductName, o.UserId }).Join(db.Users, o => o.UserId, u => u.UserId, (o, u) => new { o.ProductName, u.Username }).Where(x => x.Quantity > 2).ToList();

#	SQL Query	Entity Framework (MVC.NET)
26	SELECT Users.Username, OrderItems.ProductName, Orders.Status FROM Users JOIN Orders ON Users.UserId = Orders.UserId JOIN OrderItems ON Orders.OrderId = OrderItems.OrderId WHERE OrderItems.Price > 100	var data = db.Users.Join(db.Orders, u => u.UserId, o => o.UserId, (u, o) => new { u.Username, o.OrderId }).Join(db.OrderItems, o => o.OrderId, oi => oi.OrderId, (o, oi) => new { o.Username, oi.ProductName, o.Status }).Where(x => x.Price > 100).ToList();
27	SELECT Users.Username, OrderItems.ProductName, Orders.OrderDate FROM Users JOIN Orders ON Users.UserId = Orders.UserId JOIN OrderItems ON Orders.OrderId = OrderItems.OrderId WHERE Orders.OrderDate BETWEEN '2025-01-01' AND '2025-12-31'	var data = db.Users.Join(db.Orders, u => u.UserId, o => o.UserId, (u, o) => new { u.Username, o.OrderDate }).Join(db.OrderItems, o => o.OrderId, oi => oi.OrderId, (o, oi) => new { o.Username, oi.ProductName, o.OrderDate }).Where(x => x.OrderDate >= new DateTime(2025, 1, 1) && x.OrderDate <= new DateTime(2025, 12, 31)).ToList();
28	SELECT Users.Username, Orders.Status, OrderItems.Price FROM Users JOIN Orders ON Users.UserId = Orders.UserId JOIN	var data = db.Users.Join(db.Orders, u => u.UserId, o => o.UserId, (u, o) => new { u.Username, o.Status }).Join(db.OrderItems, o => o.OrderId, oi

#	SQL Query	Entity Framework (MVC.NET)
	OrderItems ON Orders.OrderId = OrderItems.OrderId WHERE OrderItems.Quantity > 10 AND Orders.Status = 'Shipped'	=> oi.OrderId, (o, oi) => new { o.Username, oi.Price, o.Status }).Where(x => x.Quantity > 10 && x.Status == "Shipped").ToList();
29	SELECT Orders.OrderId, Users.Username, OrderItems.ProductName FROM Orders JOIN Users ON Orders.UserId = Users.UserId JOIN OrderItems ON Orders.OrderId = OrderItems.OrderId WHERE Orders.Status = 'Completed' AND OrderItems.Price < 50	var data = db.Orders.Join(db.Users, o => o.UserId, u => u.UserId, (o, u) => new { o.OrderId, u.Username }).Join(db.OrderItems, o => o.OrderId, oi => oi.OrderId, (o, oi) => new { o.OrderId, o.Username, oi.ProductName }).Where(x => x.Status == "Completed" && x.Price < 50).ToList();
30	SELECT OrderItems.ProductName, Users.Username, Orders.OrderDate FROM OrderItems JOIN Orders ON OrderItems.OrderId = Orders.OrderId JOIN Users ON Orders.UserId = Users.UserId WHERE OrderItems.FilePath IS NOT NULL	var data = db.OrderItems.Join(db.Orders, oi => oi.OrderId, o => o.OrderId, (oi, o) => new { oi.ProductName, o.OrderDate }).Join(db.Users, o => o.UserId, u => u.UserId, (o, u) => new { o.ProductName, u.Username, o.OrderDate }).Where(x => x.FilePath != null).ToList();